

1. Introduction

The purpose of this assignment is to implement and analyze efficient algorithms for sorting and selecting elements in large integer arrays using Divide and Conquer techniques.

Three algorithms were implemented in Java: MergeSort, QuickSort, and QuickSelect. Their performance was evaluated using three metrics: execution time, number of element comparisons, and maximum recursion depth.

The algorithms were tested on arrays containing 1,000, 10,000, 100,000, and 1,000,000 elements. Three different input cases were used: random arrays, already sorted arrays, and arrays containing many duplicate values.

The experimental results were collected using five runs for each configuration. The median execution time was used to reduce the influence of individual variations in execution time. The results were saved to a CSV file and used to compare the practical performance of the implemented algorithms with their theoretical asymptotic complexity.

2. Algorithms Implementation

2.1 MergeSort

MergeSort was implemented using the Divide and Conquer approach. The input array is recursively divided into two smaller parts until the subarrays become small enough to be processed using Insertion Sort.

For small subarrays with 15 or fewer elements, Insertion Sort is used as a cutoff optimization. This reduces the overhead of recursive calls for small portions of the array.

During the merge operation, the two sorted parts are combined into one sorted part. A single auxiliary buffer is allocated once at the beginning of the sorting process and reused during all recursive calls. This avoids creating new arrays inside the recursive method.

The merge operation processes each element at most once, giving a linear $O(n)$ merge step.

The main recurrence for MergeSort is:

$$T(n) = 2T(n/2) + O(n)$$

Using the Master Theorem:

$$T(n) = O(n \log n)$$

for the best, average, and worst cases.

2.2 QuickSort

QuickSort was implemented using random pivot selection and three-way partitioning.

For each partitioning step, a random element is selected as the pivot. The array is divided into three regions:

- elements smaller than the pivot;
- elements equal to the pivot;
- elements greater than the pivot.

Three-way partitioning is especially useful for input arrays containing many duplicate values because all elements equal to the pivot are processed together.

To reduce recursion depth, the algorithm recursively processes the smaller partition first. The larger partition is processed using a loop instead of another recursive call. This technique helps prevent excessive recursion depth and reduces the risk of `StackOverflowError`.

The average-case recurrence is:

$$T(n) = 2T(n/2) + O(n)$$

when the partitions are reasonably balanced.

Therefore, the expected average complexity is:

$$O(n \log n)$$

The worst-case complexity remains:

$$O(n^2)$$

if the partitions become highly unbalanced.

2.3 QuickSelect

QuickSelect was implemented to find the k-th smallest element without completely sorting the array.

The algorithm uses the same three-way partitioning method as QuickSort. After partitioning, only the part of the array that can contain the required element is processed further.

If the required index k is inside the smaller-than-pivot region, the search continues in that region. If k is inside the greater-than-pivot region, the search continues there. If k belongs to the region containing elements equal to the pivot, the required element has been found.

Random pivot selection is used to provide good expected performance.

Unlike QuickSort, QuickSelect does not need to sort both sides of the partition because only one side can contain the required element.

Its expected average complexity is:

$O(n)$

while the worst-case complexity is:

$O(n^2)$

The implementation also checks invalid values of k and throws an `IllegalArgumentException` when the array is empty or the requested index is outside the valid range.

3. Asymptotic Analysis

Asymptotic analysis is used to describe how the running time and memory requirements of an algorithm grow as the input size increases. The main notations used in this project are Big O (O), Big Theta (Θ), and Big Omega (Ω).

3.1 Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ average
QuickSelect	$O(n)$	$O(n)$	$O(n^2)$	$O(1)$ auxiliary
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

3.2 MergeSort Recurrence

MergeSort divides the input into two approximately equal parts. Each part is processed recursively, and the merge operation requires linear time.

Therefore, its recurrence is:

$$T(n) = 2T(n/2) + O(n)$$

Using the Master Theorem:

- $a = 2$
- $b = 2$
- $f(n) = O(n)$
- $n^{(\log_b a)} = n$

Since $f(n)$ and $n^{(\log_b a)}$ have the same asymptotic order, the result is:

$$T(n) = \Theta(n \log n)$$

Therefore, MergeSort has $O(n \log n)$ time complexity in all three cases.

3.3 QuickSort Recurrence

QuickSort partitions the array around a pivot. If the pivot divides the array into approximately equal parts, the recurrence is:

$$T(n) = 2T(n/2) + O(n)$$

Using the Master Theorem gives:

$$T(n) = \Theta(n \log n)$$

This represents the balanced or average case.

However, if the pivot repeatedly produces very unbalanced partitions, the recurrence becomes approximately:

$$T(n) = T(n - 1) + O(n)$$

which results in:

$$T(n) = O(n^2)$$

Random pivot selection is used in the implementation to reduce the likelihood of consistently unbalanced partitions.

3.4 QuickSelect Complexity

QuickSelect performs the same partitioning operation as QuickSort but continues processing only the part that may contain the required element.

For a reasonably balanced partition, the amount of work can be represented approximately as:

$$T(n) = T(n/2) + O(n)$$

This gives an expected complexity of:

$$O(n)$$

The worst case occurs when the pivot repeatedly creates highly unbalanced partitions. In this situation, the complexity becomes:

$$O(n^2)$$

3.5 Insertion Sort Cutoff

Insertion Sort is used by MergeSort for subarrays containing 15 or fewer elements.

For an already sorted small subarray, Insertion Sort can operate in:

$$O(n)$$

For average and worst-case input, its complexity is:

$$O(n^2)$$

Since the cutoff is only 15 elements, using Insertion Sort for these small portions reduces recursive overhead while keeping the overall asymptotic complexity of MergeSort at $O(n \log n)$.

4. Metrics

To evaluate the practical performance of the implemented algorithms, three main metrics were collected: execution time, number of comparisons, and maximum recursion depth.

4.1 Execution Time

Execution time was measured using `System.nanoTime()` in Java.

The timer was started immediately before the algorithm was executed and stopped immediately after it finished. The difference between the two measurements was used to calculate the execution time in milliseconds.

Each experiment was performed five times. The median value of the five measurements was used as the final execution time. Using the median reduces the influence of unusually fast or slow individual runs.

4.2 Number of Comparisons

The number of element comparisons performed by an algorithm was recorded using the `Metrics` object.

A comparison is counted when the algorithm compares array elements or compares an element with the pivot during partitioning.

The comparison counter allows the algorithms to be compared independently of differences in the execution environment.

4.3 Maximum Recursion Depth

The maximum recursion depth was also recorded using the `Metrics` object.

The current recursion depth is increased when entering a recursive call and decreased when leaving it. The largest value reached during execution is stored as `maxDepth`.

This metric is particularly important for QuickSort because excessive recursion depth can lead to `StackOverflowError`. The QuickSort implementation reduces this risk by recursively processing the smaller partition and processing the larger partition using a loop.

QuickSelect is implemented iteratively, so its measured recursion depth remains 1 during the benchmark.

5. Experimental Results

The implemented algorithms were tested on four different input sizes: 1,000, 10,000, 100,000, and 1,000,000 elements.

Three input cases were used for each size:

- Random — randomly generated integer values.
- Sorted — an already sorted array.
- Duplicates — an array containing values from 0 to 9.

For each configuration, five benchmark runs were performed. The median execution time was recorded. The number of comparisons and maximum recursion depth were also collected.

5.1 Benchmark Results

Algorithm	Input Case	n	Time (ms)	Comparisons	Max Depth
MergeSort	random	1000	0,0989	9558	8
MergeSort	sorted	1000	0,0409	4236	8
MergeSort	duplicates	1000	0,0697	9191	8
QuickSort	random	1000	0,1877	16761	6
QuickSort	sorted	1000	0,0527	16423	6
QuickSort	duplicates	1000	0,0128	4760	3
QuickSelect	random	1000	0,0107	3458	1
QuickSelect	sorted	1000	0,0050	4936	1
QuickSelect	duplicates	1000	0,0115	4316	1
MergeSort	random	10000	1,6864	127140	11
MergeSort	sorted	10000	0,9124	59248	11
MergeSort	duplicates	10000	1,7464	121266	11
QuickSort	random	10000	0,7662	260004	9
QuickSort	sorted	10000	0,3870	235883	8
QuickSort	duplicates	10000	0,1478	54970	3
QuickSelect	random	10000	0,1370	60287	1
QuickSelect	sorted	10000	0,0610	67283	1
QuickSelect	duplicates	10000	0,1042	30165	1
MergeSort	random	100000	7,4165	1639809	14
MergeSort	sorted	100000	1,9030	744016	14
MergeSort	duplicates	100000	3,7312	1562182	14
QuickSort	random	100000	7,0855	3270106	10
QuickSort	sorted	100000	3,6020	3241617	11
QuickSort	duplicates	100000	1,1182	670795	2
QuickSelect	random	100000	0,8865	497002	1
QuickSelect	duplicates	100000	0,6680	670397	1
MergeSort	random	1000000	89,4045	19887347	18
MergeSort	sorted	1000000	23,6380	9071040	18
MergeSort	duplicates	1000000	41,6531	18924205	18
QuickSort	sorted	1000000	40,0805	37160323	13
QuickSort	duplicates	1000000	10,9693	7101427	2
QuickSelect	random	1000000	9,5059	4553180	1
QuickSelect	sorted	1000000	2,6966	3102435	1
QuickSelect	duplicates	1000000	8,2933	2200269	1

5.2 General Observations

The results show that execution time increases as the input size grows, while the number of comparisons also increases.

MergeSort shows stable recursion depth for arrays of the same size because the array is consistently divided into approximately equal parts. Its maximum depth increases gradually as the input size increases.

QuickSort shows lower recursion depth than a straightforward recursive implementation because the algorithm recursively processes the smaller partition and handles the larger partition iteratively.

The duplicate-heavy input produces particularly low QuickSort recursion depths. This is a result of the three-way partitioning, which places all elements equal to the pivot into one partition.

QuickSelect performs fewer operations than the complete sorting algorithms because it only processes the part of the array that can contain the requested element. Its measured recursion depth is 1 because the implementation processes the selection iteratively.

6. Discussion

The experimental results demonstrate how the characteristics of the input data affect the practical performance of the implemented algorithms.

6.1 MergeSort

MergeSort provides predictable performance because the array is always divided into approximately equal parts. Its maximum recursion depth increases gradually with the input size: from 8 for 1,000 elements to 18 for 1,000,000 elements.

The number of comparisons depends on the input data. For example, for 1,000,000 elements, MergeSort performed 9,071,040 comparisons on the sorted input and 19,887,347 comparisons on the random input.

The execution time was also lower for sorted input. This is partly caused by the merge operation requiring fewer element comparisons when the input portions are already ordered.

6.2 QuickSort

QuickSort showed different performance depending on the input case. The use of a random pivot helps avoid consistently unbalanced partitions, while three-way partitioning makes the algorithm particularly effective when many duplicate values are present.

For 1,000,000 duplicate elements, QuickSort required 10.9693 ms and had a maximum depth of only 2. In comparison, the random input required 83.8754 ms and had a maximum depth of 12.

This demonstrates the benefit of three-way partitioning for duplicate-heavy arrays.

The recursion-depth optimization also keeps the maximum depth relatively small even for large arrays. The algorithm recursively processes the smaller partition and handles the larger partition iteratively.

6.3 QuickSelect

QuickSelect generally requires less work than a complete sorting algorithm because it does not need to sort the entire array.

For example, with 1,000,000 random elements, QuickSelect required 9.5059 ms and performed 4,553,180 comparisons, while QuickSort required 83.8754 ms and performed 38,310,193 comparisons.

The measured maximum depth is 1 because the current QuickSelect implementation uses an iterative loop rather than recursive calls.

6.4 Effect of Input Size

Increasing the input size from 1,000 to 1,000,000 elements demonstrates the growth predicted by the asymptotic complexity analysis.

The sorting algorithms require substantially more comparisons as the input size increases. However, the growth is consistent with the expected $O(n \log n)$ behavior for MergeSort and the expected average-case behavior of randomized QuickSort.

QuickSelect shows a slower growth in the amount of work compared with complete sorting because it only processes the partition containing the requested element.

Overall, the experimental results are consistent with the theoretical complexity of the implemented algorithms, while also showing that input characteristics such as sortedness and the number of duplicate values can significantly affect practical performance.

7. Conclusion

This assignment provided practical experience with Divide and Conquer algorithms and asymptotic analysis. Three algorithms were implemented in Java: MergeSort, QuickSort, and QuickSelect.

The algorithms were tested on arrays ranging from 1,000 to 1,000,000 elements using random, sorted, and duplicate-heavy input data. Execution time, number of comparisons, and maximum recursion depth were collected during the experiments.

The results demonstrated the practical differences between the algorithms. MergeSort provided predictable $O(n \log n)$ sorting performance, while QuickSort benefited from random pivot selection and three-way partitioning, especially for arrays containing many duplicate values. QuickSelect avoided sorting the entire array and therefore required less work when only one order statistic was needed.

The assignment also demonstrated the importance of implementation details. Reusing the MergeSort buffer, using an insertion-sort cutoff, selecting random pivots, using three-way partitioning, and controlling recursion depth all affected the practical behavior of the algorithms.

Overall, the implementation and experiments showed how theoretical asymptotic complexity relates to the actual performance of algorithms on different types and sizes of input data.